

GRUPO I -

1) `static String mapaDePrecos(Planta[][] horto, Pair<Integer,Integer>[] v)`

Objetivo: construir uma string com uma linha por cada par (i,j) em v , **mas só se** (i,j) forem índices válidos da matriz.

Para cada par válido:

- Se `horto[i][j] == null` → escrever: `"(i , j ): Nao existe"`
- Se `horto[i][j] != null` → escrever: `"(i , j ): " + nome + " " + preco`
- Se i ou j fora dos limites → **ignorar** (não escreve nada)

Isto está descrito no enunciado (Grupo I, Q1).

Passo a passo (lógica)

1. Criar `StringBuilder sb`
2. Percorrer todos os pares do vetor v
3. Para cada par, obter $i = \text{first}()$, $j = \text{second}()$
4. Verificar limites:
 - $0 \leq i < \text{horto.length}$
 - $\text{horto.length} > 0$ e $0 \leq j < \text{horto}[i].\text{length}$ (ou `horto[0].length` se matriz “certinha”)
5. Se inválido → `continue`
6. Se válido:
 - Se `horto[i][j] == null` → append linha “Nao existe”
 - Senão → append linha com `nome()` e `preco()`
7. No fim devolver `sb.toString()`

```
static String mapaDePrecos(Planta[][] horto, Pair<Integer, Integer>[] v) {
    StringBuilder sb = new StringBuilder();

    for (Pair<Integer, Integer> par : v) {
        int i = par.first();
        int j = par.second();

        // validar índices
        if (i < 0 || i >= horto.length) continue;
        if (horto[i] == null) continue; // por segurança
        if (j < 0 || j >= horto[i].length) continue;

        sb.append("(").append(i).append(" , ").append(j).append(" ): ");

        Planta p = horto[i][j];
        if (p == null) {
            sb.append("Nao existe");
        } else {
            sb.append(p.nome()).append(" ").append(p.preco());
        }
        sb.append("\n");
    }
    return sb.toString();
}
```

2(a)

- objetivo = 5
- produtos = [Vinho, Bombons, Mel]
- completo = false (só tem 3/5)

Imprime: `Objetivo = 5; Produtos = [Vinho, Bombons, Mel]`

2(b)

- objetivo = 3
- produtos = [Nozes, Bacalhau, Gin]
- completo = true (agora tem 3/3)
- doadores = [Maria]

Imprime: `Objetivo = 3; Produtos = [Nozes, Bacalhau, Gin]; doadores = [Maria]`

2(c)

- objetivo = 5
- produtos = [Pao, Bolo, Croissant, Empada]
- completo = false (4/5)
- doadores = [Rui, Rita]
 - "Croissant", "Rita" adiciona Rita
 - "Empada" usa o doador inicial Rui (não adiciona novo doador)

Imprime: Objetivo = 5; Produtos = [Pao, Bolo, Croissant, Empada]; doadores = [Rui, Rita]

GRUPO II - 1) Classe abstrata Leilao

- atributos: dataLimite, taxaComissao, aLeilao
- métodos:
 - boolean terminou() → now é igual ou depois da dataLimite
 - int comissao() → (valor obtido no resultado) * taxaComissao / 100
 - Map<String,Integer> valores() → tabela descrição→preço dos elementos a leiloar
 - fazerLicitacao e resultado são abstratos

Passo a passo da implementação

1. Guardar os 3 atributos no construtor (copiar a lista para segurança)
2. terminou():
 - LocalDateTime agora = LocalDateTime.now()
 - terminou se agora.isAfter(dataLimite) ou agora.equals(dataLimite)
3. valores():
 - criar Map<String,Integer>
 - para cada elemento e em aLeilao: map.put(e.descricao(), e.preco())
4. comissao():
 - Pair<String,Integer> r = resultado()
 - int obtido = r.second()
 - return obtido * taxaComissao / 100

```
public abstract class Leilao implements Avaliavel {

    private LocalDateTime dataLimite;
    private int taxaComissao;
    private List<ElementoALeiloar> aLeilao;

    public Leilao(LocalDateTime dataLimite, int taxaComissao, List<ElementoALeiloar> aLeilao) {
        this.dataLimite = dataLimite;
        this.taxaComissao = taxaComissao;
        this.aLeilao = new ArrayList<>(aLeilao);
    }

    public int taxaComissao() { return taxaComissao; }
    public List<ElementoALeiloar> aLeilao() { return new ArrayList<>(aLeilao); }

    public boolean terminou() {
        LocalDateTime agora = LocalDateTime.now();
        return agora.equals(dataLimite) || agora.isAfter(dataLimite);
    }

    public double comissao() {
        int obtido = resultado().second();
        return obtido * (taxaComissao / 100.0);
    }

    @Override
    public Map<String, Integer> valores() {
        Map<String, Integer> res = new HashMap<>();
        for (ElementoALeiloar e : aLeilao) {
            res.put(e.descricao(), e.preco());
        }
        return res;
    }

    public abstract void fazerLicitacao(String licitador, int valor);
    public abstract Pair<String, Integer> resultado();
}
```

2) LeilaoCego

- guardar **todas** as licitações
- `resultado()` = licitação de maior valor
- `retirarLicitacao(licitador)` = remove a licitação desse licitador

Passo a passo

1. Atributo: `List<Pair<String,Integer>> licitacoes`
2. `fazerLicitacao`:
 - se leilão terminou → ignorar (ou não registrar)
 - senão: `licitacoes.add(new Pair<>(licitador, valor))`
3. `resultado`: percorrer `licitacoes` e escolher a maior (se lista vazia → devolver `(null,0)`)
4. `retirarLicitacao`: remover o primeiro par cujo `first().equals(licitador)`

```
public class LeilaoCego extends Leilao {

    private List<Pair<String, Integer>> licitacoes;

    public LeilaoCego(LocalDate dataLimite, int taxaComissao, List<ElementoALeiloar> aLeilao) {
        super(dataLimite, taxaComissao, aLeilao);
        this.licitacoes = new ArrayList<>();
    }

    @Override
    public void fazerLicitacao(String licitador, int valor) {
        if (terminou()) return;
        licitacoes.add(new Pair<>(licitador, valor));
    }

    @Override
    public Pair<String, Integer> resultado() {
        Pair<String, Integer> best = new Pair<>(null, 0);
        for (Pair<String, Integer> p : licitacoes) {
            if (p.second() > best.second()) best = p;
        }
        return best;
    }

    public void retirarLicitacao(String licitador) {
        for (int i = 0; i < licitacoes.size(); i++) {
            if (licitacoes.get(i).first().equals(licitador)) {
                licitacoes.remove(i);
                return;
            }
        }
    }
}
```

3) LeilaoDireto

- atributo `minPretendido`
- atributo `ultimaLicitacao : Pair<String,Integer>`
- só regista licitação se for **maior** que a última
- `resultado()`:
 - se `ultimaLicitacao.valor >= minPretendido` → devolver `ultimaLicitacao`
 - senão → devolver `(null,0)`
- redefinir:
 - `terminou()` → termina se (data limite alcançada) **ou** (ultima >= minPretendido)
 - `comissao()` → se resultado for 0, comissão = taxa sobre **minPretendido**; senão igual ao da super

Passo a passo

1. Inicializar `ultimaLicitacao = new Pair<>(null,0)`
2. `fazerLicitacao`:
 - se terminou → return
 - se valor > ultima.second → ultima = (licitador, valor)

3. **resultado** conforme regra
4. **terminou** com OR
5. **comissao** com caso especial

```
public class LeilaoDireto extends Leilao {

    private int minPretendido;
    private Pair<String, Integer> ultimaLicitacao;

    public LeilaoDireto(LocalDate dataLimite, int taxaComissao,
                        List<ElementoALeiloar> aLeilao, int minPretendido) {
        super(dataLimite, taxaComissao, aLeilao);
        this.minPretendido = minPretendido;
        this.ultimaLicitacao = new Pair<>(null, 0);
    }

    @Override
    public void fazerLicitacao(String licitador, int valor) {
        if (terminou()) return;
        if (valor > ultimaLicitacao.second()) {
            ultimaLicitacao = new Pair<>(licitador, valor);
        }
    }

    @Override
    public Pair<String, Integer> resultado() {
        if (ultimaLicitacao.second() >= minPretendido) return ultimaLicitacao;
        return new Pair<>(null, 0);
    }

    public int minimoPretendido() { return minPretendido; }

    public Pair<String, Integer> ultimaLicitacao() { return ultimaLicitacao; }

    @Override
    public boolean terminou() {
        return super.terminou() || ultimaLicitacao.second() >= minPretendido;
    }

    @Override
    public double comissao() {
        Pair<String, Integer> r = resultado();
        if (r.second() == 0) {
            return minPretendido * (taxaComissao() / 100.0);
        }
        return super.comissao();
    }
}
```

4)

(b) Implementar **imprimeResultados**

Pede: para cada leilão de uma coleção, imprimir:

- o resultado (licitador e valor)
- a comissão

```
static void imprimeResultados(Iterable<? extends Leilao> leiloes) {
    for (Leilao l : leiloes) {
        Pair<String, Integer> r = l.resultado();
        System.out.println("Resultado: licitador=" + r.first() + ", valor=" + r.second());
        System.out.println("Comissao: " + l.comissao());
    }
}
```